



PySPEDAS講習会

名古屋大学宇宙地球環境研究所
太陽圏サイエンスセンター



- PySPEDASを使ってあらせ衛星データやその関連データをどうやって読み込み・プロット・操作するか、を学ぶ。
- この講習では、Pythonの環境構築、PySPEDASのセットアップに注力した**基礎パート**と、実際の衛星データの解析を行う**応用パート**をハンズオン形式で行う予定である。

基礎パート

0. インストール・セットアップ



PySPEDASを使うためには、PythonをPC or サーバーにインストールし、（できれば）PySPEDAS用の仮想環境を構築する必要がある：

**仮想環境はマストではないが、システムのPythonに変更を加えると色々と面倒なので、仮想環境の構築を推奨する

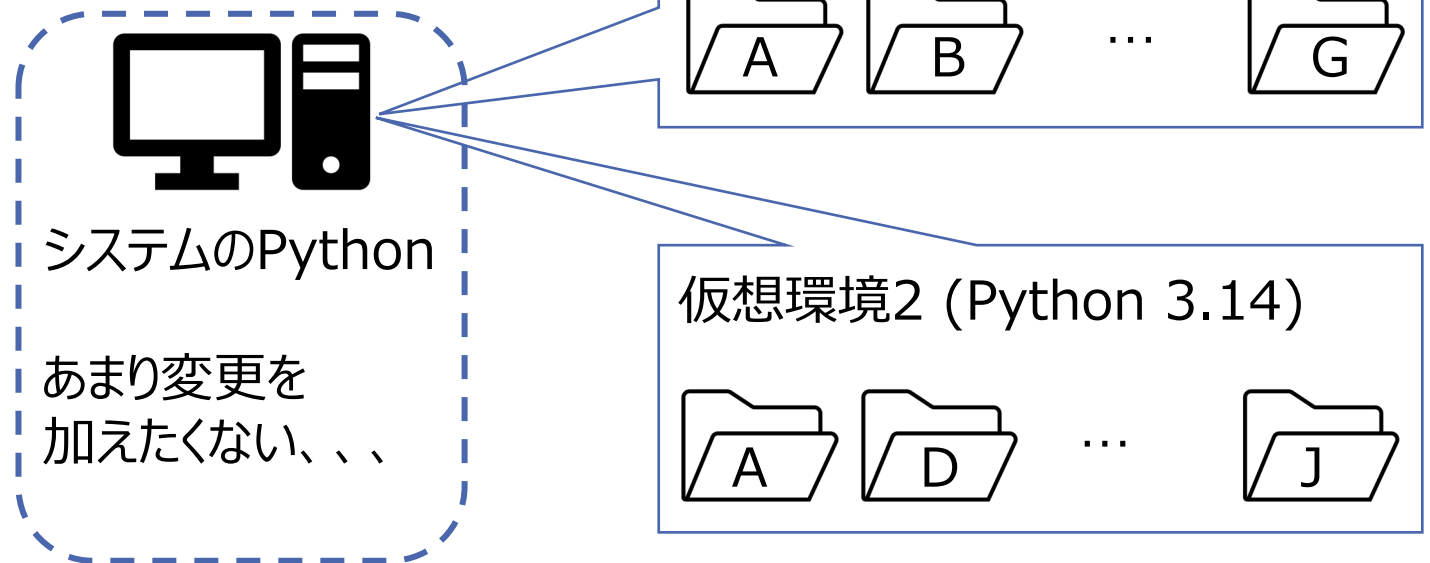
- 主な仮想環境の管理ツール

- venv : Pythonの標準仮想環境作成ツール（自身のPythonに合わせて手軽に仮想環境を作成できる）
- pyenv+venv : pyenvを使うことでPython自体のバージョン管理も可能
- miniforge : conda-forge(condaのオープンソース版)のパッケージ管理ツール
 - ライセンス改訂により**Anacondaは有償化されたので要注意**
- uv : 最近流行りのパッケージ管理ツール、Python自体のバージョン管理も可能
 - 本講習では、uvを使った環境構築を行う（PySPEDAS本家はvenvを推奨）

仮想環境：1台のPCの中で、独立した“仮想的”なPythonの実行環境(やパッケージ管理)を構築できる仕組み

仮想環境を使う主な理由

- ライブラリの依存関係の競合の防止
- 開発環境の共有のしやすさ(再現性の確保)
- システムのPythonに変更を加えなくて済む



uvのインストール & 環境構築 (PySPEDAS用の仮想環境を構築済みの人は飛ばしてもOK)



1. uvのインストール

- macOS/Linux(WSL)の場合ターミナルで以下のコマンドを実行する。

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

- Windowsの場合powershellで以下のコマンドを実行する。

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

2. 仮想環境の構築

```
uv init pyspedas_tutorial -p 3.13
```

- `pyspedas\_tutorial` という名前の仮想環境を作成します。
仮想環境の名前は任意に設定できるので自分にとってわかりやすいプロジェクト名にしてください
- -p *.*で任意のPythonバージョンを指定できます。

PySPEDASに必要なパッケージのインストール



- ターミナル/PowerShellで以下のコマンドを実行

```
cd <path_to_environment>/pyspedas_tutorial
uv add ipykernel
uv add pyspedas
```

- 詳細はPySPEDAS本家のGithubを参照
<https://github.com/spedas/pyspedas>
- uvを使う場合はpip installの代わりにuv addを使うこと(または uv pip install)。
**何も指定せずにpip installをしてしまうと ローカルのPythonにパッケージをインストールしてしまうので注意
- あらせデータの読込・解析のPySPEDAS向け追加パッケージ:
ERG plug-in tool for PySPEDAS
https://github.com/ergsc-devel/pyspedas_plugin
を追加する場合は以下のコマンドも実行
(*下記のコマンドはgitをinstallしていない場合はうまく動かないので注意: gitをインストールできない場合はgithubよりzip fileをダウンロードし、erg_plugin-masterに入っているergpyspedasのディレクトリを~/pyspedas_tutorial/.venv/lib/site-packages/配下にコピー&ペーストしてください)

```
uv add git+https://github.com/ergsc-devel/pyspedas\_plugin
```

PySPEDASのためのコード(プログラム)をどうやって書くか?



文法チェック、色分け、括弧閉じチェック、字下げ・改行などの
フォーマット一括適用、など

```
1 import pyspedas # import pyspedas modules
2 import ergpyspedas # import modules of the bleeding-edge of the ERG-SC plugin
3 from pyspedas import tplot # shortcut for pyplot.tplot function
4 import matplotlib.pyplot as plt # shortcut for matplotlib.pyplot module
5 # show the path to the imported PySPEDAS package
6 # print(pyspedas.__file__)
7 # print(ergpyspedas.__file__)

[3] ✓ 0.0s Python

1 # import a function of mgf in (package directory)/pyspedas/projects/erg/satellite/erg/mgf/mgf.py
2 from ergpyspedas.erg import mgf
3
4 trange = ['20230423', '20230425'] # trange is a list type specifying the period for data loading
5 uname = ''
6 # username is a string required for downloading recent data. For public data, this is a empty string ''.
7 passwd = ''
8 # password is a string required for downloading recent data. For public data, this is a empty string ''.
9 var_mgf = mgf(level='l2', datatype='8sec', coord='', trange=trange, uname=uname, passwd=passwd) #var_mgf contains names of loaded tplot-variables of MGF

[10] ✓ 2.4s Python
```

- VS code + Python拡張機能(Python, Python Environment, Jupyterなど)を使うのがおすすめ (仮想環境を選択しやすい、さまざまな言語に対応してるので便利)
- VS code: <https://code.visualstudio.com>

PySPEDASのためのコード(プログラム)をどうやって書くか?



- Pythonプログラムを動かす手順:

1) 仮想環境の起動 → 2) editor(or統合環境)でコードの編集・(シェル上で)実行

1. 仮想環境の起動 (~/tutorial_20260916での作業を想定)

- shell上で `mkdir ~/tutorial_20260916 && cd ~/tutorial_20260916` を実行してください)

MacOS/Linuxの場合

```
source path/to/project/.venv/bin/activate
```

Windowsの場合(powershell上で)

```
path/to/project/.venv¥Scripts¥Activate.ps1
```

**uvではない環境を使っている場合はご自身の構築した仮想環境を起動してください
(uvは中でvenvを使っているのでvenvと操作感はほとんど同じです)

PySPEDASのためのコード(プログラム)をどうやって書くか?



- Pythonプログラムを動かす手順:

1) 仮想環境の起動→ 2) editor(or統合環境)でコードの編集・(シェル上で)実行

2. editorの起動 (~/tutorial_20260916での作業を想定)

VS codeを使う場合 (~/tutorial_20260916にて)以下のコマンドを実行

```
code .
```

Jupyter notebook (Jupyter Lab)を使う場合 (まずはjupyterのインストールから)

```
uv add jupyter (必要に応じてuv add jupyter labも)  
uv run --with jupyter notebook
```

Editor起動後はeditor上でipynbファイルを実行できます。



- PySPEDASは2025年11月にv2.*系にMajor Update
 - 一見、pytplotが廃止されたことで、v1.*系との互換性が切れたように見える
→実際のところは、pytplotがpyspedasに吸収されたので、自身のコードにて
pytplot→pyspedasと置換すれば問題ない
- 生成AIに関わる動向
 - 最近、公式がAIEージェント用のAPIをリリース → 理論的には自然言語ベースの解析が可能に
 - SPEDASはオープンソースなので、現状でも生成AIがソースコードを読みについてかなりの精度の解析プログラムを作成してくれる

基礎パート

1. SPEDASとは?

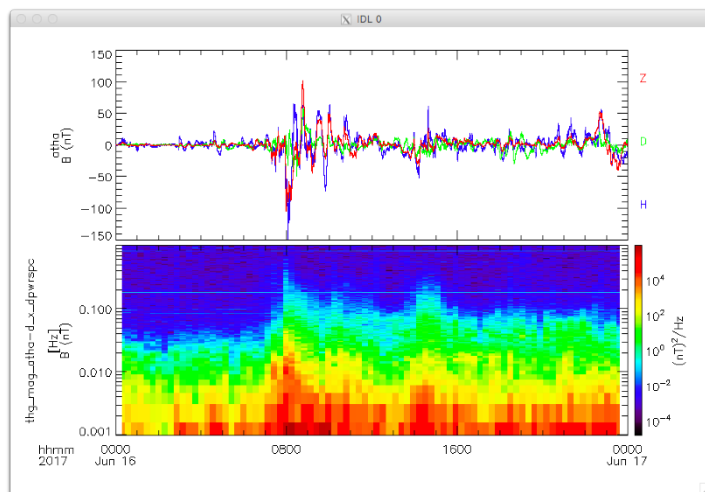
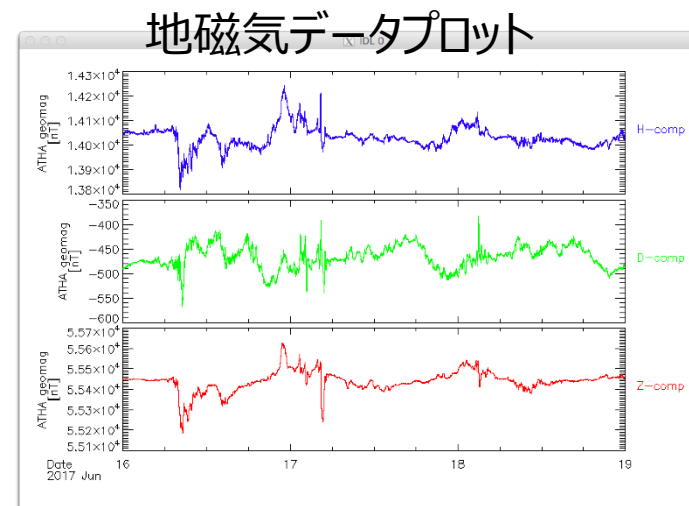
Arase



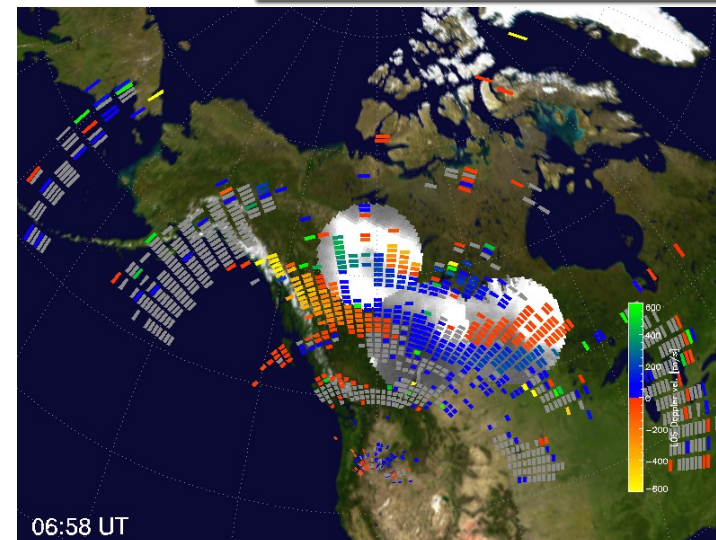
SPEDASで何ができるか？



- あらせ衛星や他衛星・地上データの読込
- 時系列データプロット
- データへの各種フィルターの適用
- 周波数解析
- 地上・電離圏へのマッピング
- ...

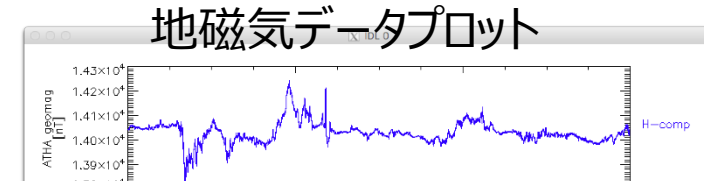


地磁気データの周波数スペクトル



SuperDARN (大型電離圏短波レーダー)と全天カメラのデータを地図上に描画

- あらせ衛星や他衛星・地上データの読込
- 時系列データプロット



必ずしもPySPEDASを使わなければ解析が出来ないわけではない
(numpy、scipyなど便利な解析パッケージが存在)
→ PySPEDASはデータローダーとしてはかなり優秀

- PySPEDASの使用例
 1. データのロードはPySPEDASで行い、解析は自身のプログラムで行う
 2. データのロード+データ整理まではPySPEDASで行い、詳細な解析は自身のプログラムで行う
 3. データのロードから解析までを全てPySPEDASで行う

地磁気データの周波数スペクトル

06:58 UT

SPEDAS / PySPEDAS Analysis tool



SPEDAS (<https://github.com/spedas>)

Space Physics Environment Data Analysis Software

44 followers

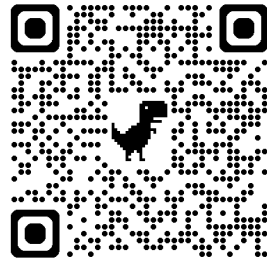
<http://www.spedas.org>

Pinned

pyspedas Public

Python-based Space Physics Environment Data Analysis Software

Python 188 71



Repositories

Find a repository...

Type

Language

Sort

bleeding_edge Public

IDL-based Space Physics Environment Data Analysis Software (bleeding edge)

IDL 7 1 92 0 Updated 3 days ago

pyspedas Public

Python-based Space Physics Environment Data Analysis Software

Python 188 MIT 71 155 1 Updated 5 days ago

IDL version

Python version

bleeding_edge Public

master 2 Branches 0 Tags

github-actions[bot]	Mirror SPEDAS bleeding_edge.zip (d720f3092dd) on 2026-02-02 1...	b703efe · 3 days ago	1273 Commits
.github/workflows	Filter *.html changes out of log output, since they all change...	5 months ago	
idl	Mirror SPEDAS bleeding_edge.zip (d720f3092dd) on 2026-0...	3 days ago	
.gitattributes	overriding default language detection for .pro files	5 years ago	
.gitignore	updating to current bleeding edge	3 years ago	
LICENSE.txt	Add files via upload	6 months ago	
README.md	Create README.md	6 months ago	

README License

Github Mirror of IDL SPEDAS

This repository contains a snapshot of the IDL SPEDAS "bleeding edge" source code repository. It is periodically refreshed from the official repository, but we do not recommend installing it from here. Instead, go to one of the official download locations, to ensure that you're getting the latest official release, or the latest unofficial "bleeding edge" snapshot:

SPEDAS Wiki: https://spedas.org/wiki/index.php?title=Downloads_and_Installation

THEMIS software page: <https://themis.ssl.berkeley.edu/software.shtml>

The IDL SPEDAS developers do monitor the Issues page for this repo, so if you'd like to report a bug, or request an enhancement, please do open an issue.

PySPEDAS

build passing coverage 90% pypi v2.0.7 status stable license MIT
repo status Active

The Python-based Space Physics Environment Data Analysis Software (PySPEDAS) framework supports multi-mission, multi-instrument retrieval, analysis, and visualization of heliophysics time series data.

Projects Supported

- [Advanced Composition Explorer \(ACE\)](#)
- [Akebono](#)
- [Arase \(ERG\)](#)
- [Cluster](#)
- [Colorado Student Space Weather Experiment \(CSSWE\)](#)
- [Communications/Navigation Outage Forecasting System \(C/NOFS\)](#)
- [Deep Space Climate Observatory \(DSCOVR\)](#)
- [Dynamics Explorer 2 \(DE2\)](#)

基礎パート

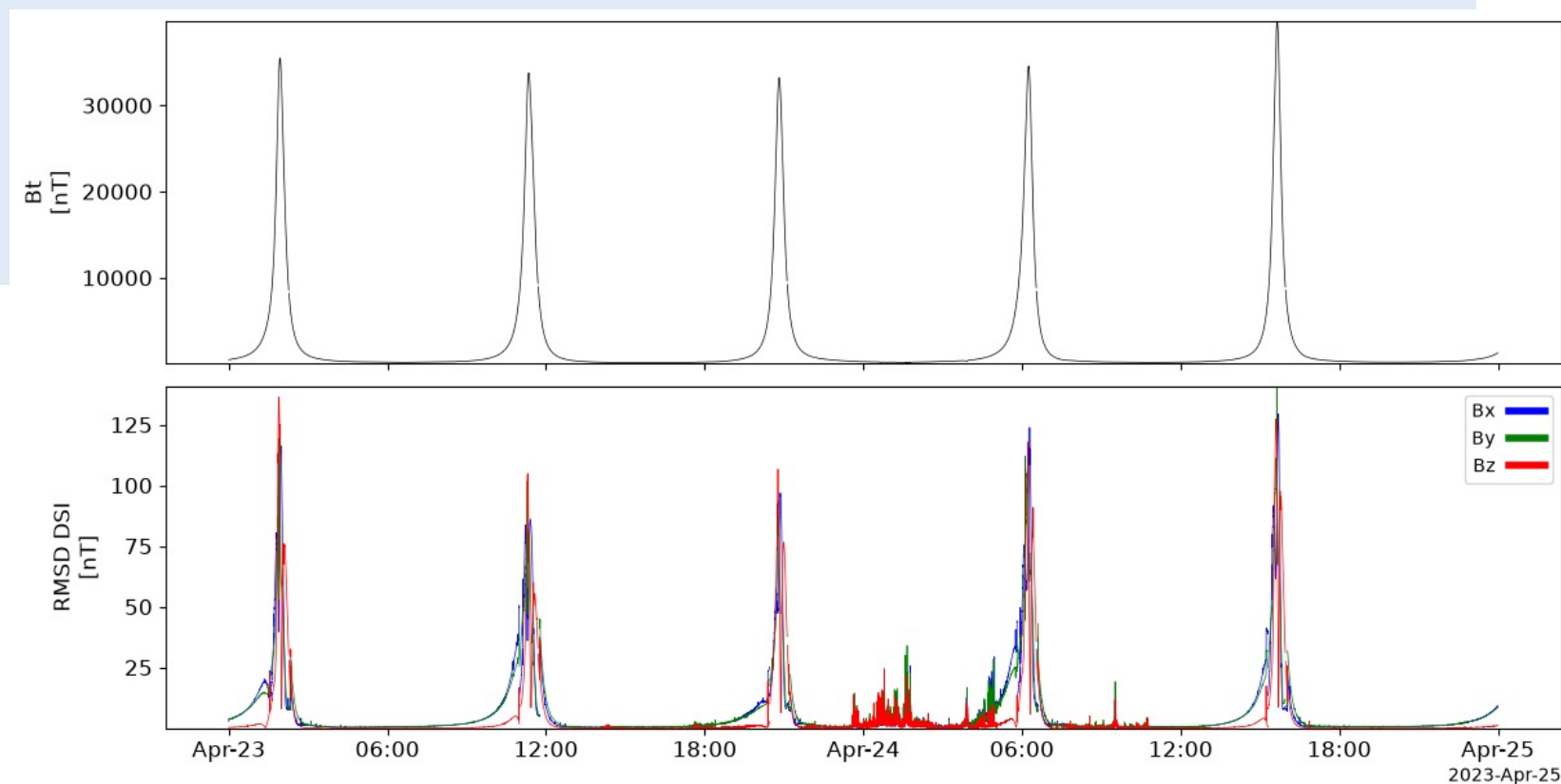
2. tplot と tplot変数

PySPEDASによるデータ処理の基本パターン



一番シンプルな手順は下記のような感じ

1. 作業ディレクトリに移動 (+必要に応じて仮想環境を起動)
2. Jupyter notebookを起動
3. データの日付・時刻幅を設定
4. データを読み込む
5. (必要ならデータを加工)
6. データをプロット

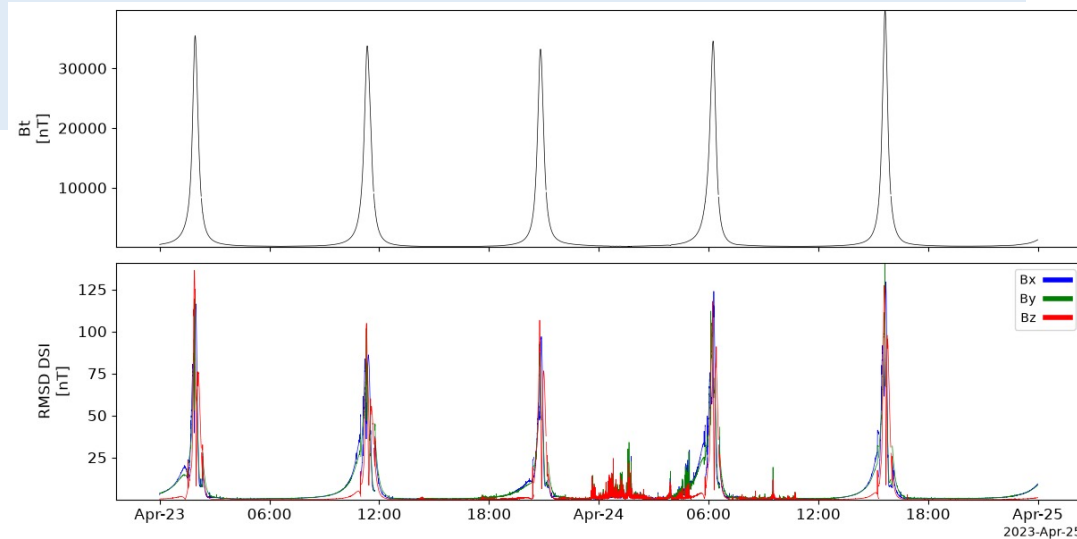


SPEDASによるデータ処理の基本パターン (続き)



実際のコマンド例:

```
import pyspedas
import ergpyspedas
from ergpyspedas.erg import mgf
trange = ['20230423', '20230425']
var_mgf = mgf(level='l2', datatype='8sec', coord='', trange=trange)
(必要ならここでデータを加工)
pyspedas.tplot(['erg_mgf_l2_magt_8sec', 'erg_mgf_l2_mag_8sec_dsi'])
```





`pyspedas.timespan('timestr', dt, units)`

`timestr` : 設定したい日時幅の開始時刻(UTC)を
'yyyy-mm-dd/hh:mm:ss'形式の文字列で指定

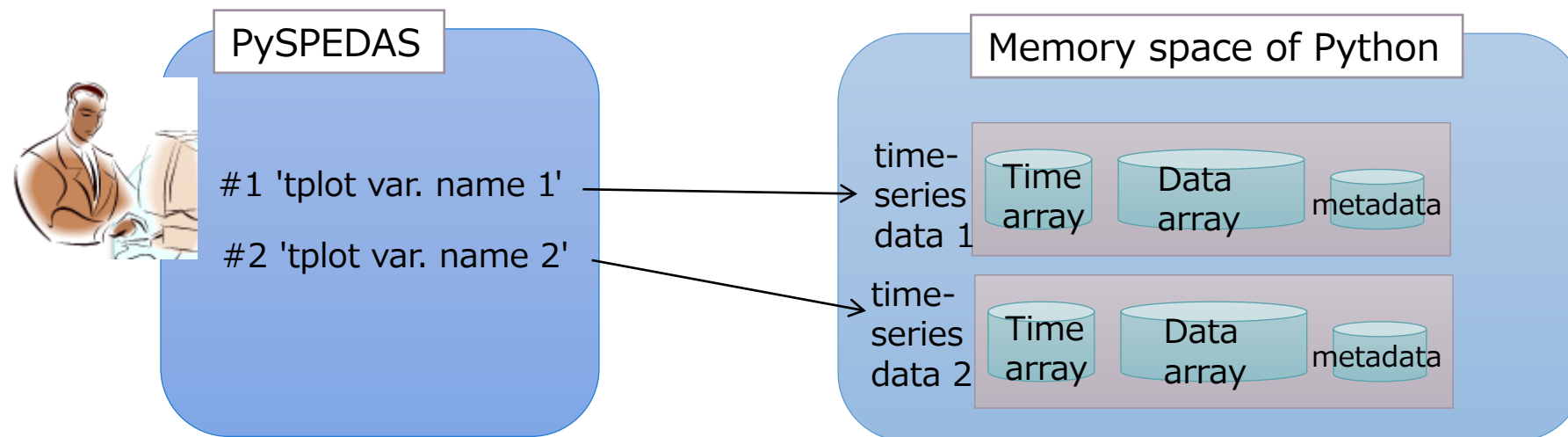
`dt` : 時間幅、`units` : 時間幅の単位を文字列で指定(e.g., 'hours')

例えば2017-05-27/09:00:00 UTCから6時間分の範囲なら

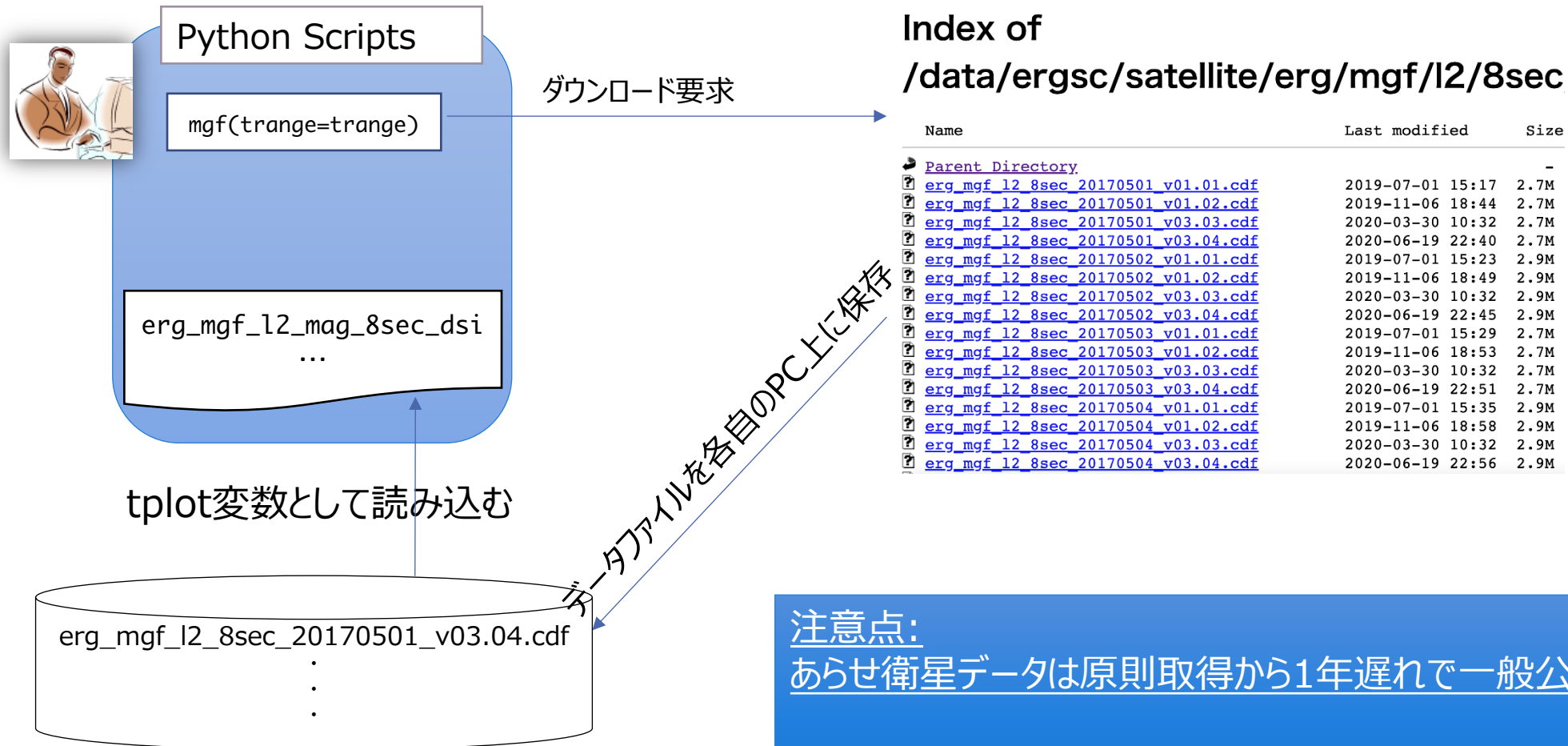
`pyspedas.timespan('2017-05-27/09:00', dt=6, units='hours')`

- `pyspedas.project.omni.load()`を使うと、OMNIデータをロードすることができます。ERGのロード関数と`pyspedas.project.omni.load()`を使って、2023/04/23から2023/04/27までのデータをロードしてみましょう。
- `tplot`を使ってERG衛星のデータ2種類+軌道と、上流の太陽風速度、IMFの絶対値、地磁気指数を一枚の図にプロットしてみましょう。
 - ✓ただし、太陽風データと地磁気指数についてどの`tplot`変数を指定すればよいかは、`pyspedas.tplot_names()`で確認すること。
 - ✓また、プロットは期間は2023/04/23から2023/04/27までを俯瞰したプロットと、この期間のうち自分が気になる12時間分を拡大したプロットの、2種類のプロットを描画してください。
- OMNIデータ:
地球周辺（近地球宇宙空間）の太陽風や惑星間磁気-interplanetary magnetic field (IMF)-の観測データを複数衛星から統合したデータベース
https://omniweb.gsfc.nasa.gov/html/ow_data.html

- 前ページの'erg_mgf_l2_mag_8sec_dsi' のようなものは ***tplot*変数 (*tplot variable*)** と呼ばれる。
- "tplot変数"は、**通し番号付きのtplot変数名**(文字列)と、実際の**メタデータ付き時系列データの配列**を結びつけるものである。
(Python中での実体はndarray形式 or xarray形式の配列)



例：MGFデータ読込ルーチンが実際には何をしているか



注意点:

あらせ衛星データは原則取得から1年遅れで一般公開

直近の1年以内のデータにアクセスするには、
ワーキンググループID/パスワードが必要。

tplot変数の中のデータ構造体にアクセスする

– get_data –



```
data = pyspedas.get_data('erg_mgf_l2_mag_8sec_gsm', xarray = False)
```

data: これにtplot変数のデータ配列が入る

xarray(bool): False → ndarray形式の配列、 True → xarray形式の配列を取得

metadata(bool): True → メタデータの配列を取得。

****ただしxarray配列は、もともとデータ、メタデータ両方を含んだ配列になっている**

```
metadata = pyspedas.get_data('erg_mgf_l2_mag_8sec_gsm', metadata = True)
```

```
Time: [1.68212160e+09 1.68212160e+09 1.68212161e+09 ... 1.68255357e+09
1.68255358e+09 1.68255359e+09]
Size: 53555

Data: [[ 130.88478 -320.36 40.291492]
 [ 130.82204 -319.85028 40.474075]
 [ 130.86896 -319.40616 40.459797]
 ...
 [ 461.83304 -16620.145 9225.124 ]
 [ 639.3126 -16931.568 9158.393 ]
 [ 825.30255 -17243.72 9086.227 ]]
Shape: (53555, 3)
```

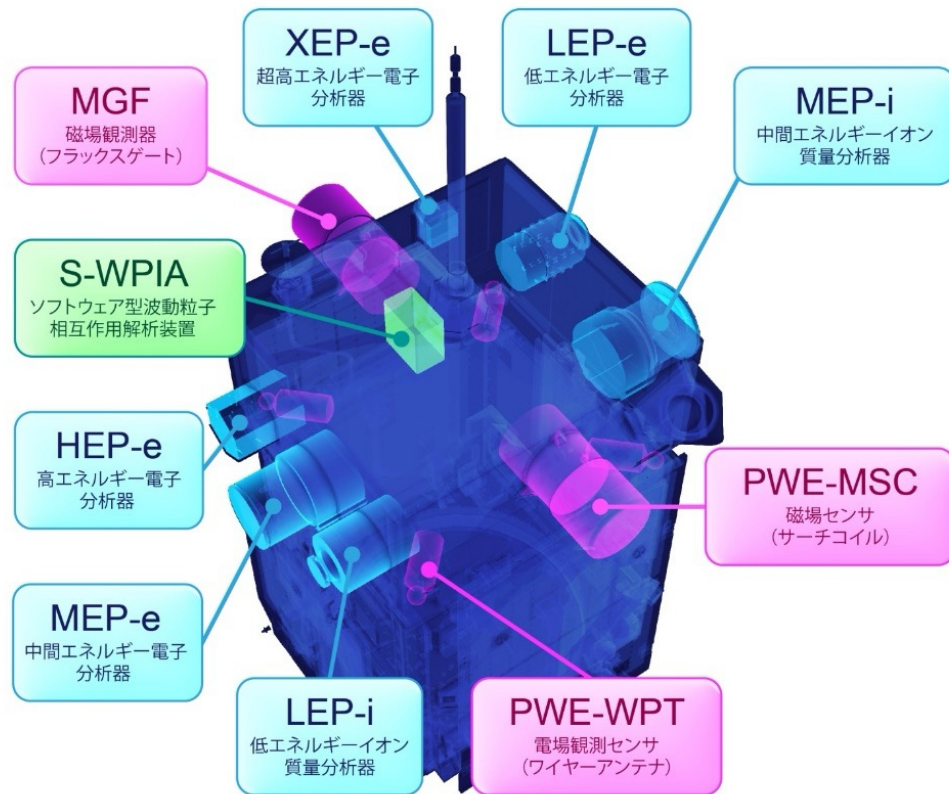
“**get_data**”によってtplot変数の中のデータを吸い出してPythonの配列（↑の例では“data”）として保存できる。それにより、データの中身に普通の配列
“**data.times**” or “**data.y**” としてアクセスできるようになる。

```
dict_keys(['CDF', 'data_att', 'plot_options'])
CDF
{'FIELDNAM': 'B in GSM', 'CATDESC': '8 sec resolution B in GSM coordinates', 'VALIDMIN': '-99999.999', 'VALIDMAX': '99999.999', 'SCALEMIN': '-99999.999', 'SCALEMAX': '99999.999', 'UNITS': 'nT', 'UNIT_PTR': 'unit_mag_8sec', 'DEPEND_0': 'epoch_8sec', 'LABLAXIS': 'B GSM', 'LABL_PTR_1': 'label_mag_8sec', 'MONOTON': 'false', 'FORMAT': 'F10.3', 'FILLVAL': '-1.0E+31', 'VAR_TYPE': 'data', 'AVG_TYPE': 'standard', 'DISPLAY_TYPE': 'time_series', 'SCALETYP': 'linear'}
{'PROJECT': ['ERG>Exploration of Energization and Radiation in Geospace'], 'DISCIPLINE': ['Space Physics>Magnetospheric Science'], 'SOURCE_NAME': ['ARASE(ERG)>Inner Magnetosphere'], 'DATA_TYPE': ['l2_mgf>level 2 spin-averaged magnetic field data'], 'DESCRIPTOR': ['MGF>Magnetic Field Experiment'], 'DATA_VERSION': ['04', '06'], 'DOI': ['10.34515/DATA.ERG-06001'], 'TITLE': ['Level 2 magnetic field data obtained by the Magnetic Field Experiment (MGF) instrument onboard the ERG satellite'], 'TEXT': ['L2L Geospace v11.0 and ...']}
```

元データに付随していたメタデータやプロット属性の情報は辞書形式で保存され、“CDF”, “data_att” などのキーワードを指定して詳細を確認できる。



- ERGプロジェクト全体についての論文
 - Miyoshi et al., EPS, 2018a
- ERGサイエンスセンターについての論文
 - Miyoshi et al., EPS, 2018b



- Orbit (predict/definitive, L2/L3)
- Attitude (L2)
- MGF (pre/L2)
 - Matsuoka et al., EPS, 2018
- PWE (pre/L2/L3)
 - Kasahara et al., EPS, 2018
 - Kumamoto et al., EPS, 2018
 - Ozaki et al., EPS, 2018
 - Matsuda et al., EPS, 2018
 - Kasaba et al., EPS, 2017
- XEP (L2/L3/L4)
- HEP (HEP-H/HEP-L, pre/ L2/L3/L4)
 - Mitani et al., EPS, 2018
- MEPi(pre/L2/L3/L4)
 - Yokota et al., EPS, 2017
- MEPe(pre/L2/L3/L4)
 - Kasahara et al., EPS, 2018
- LEPi(pre/L2/L3/L4)
 - Asamura et al., EPS, 2018
- LEPe(pre/L2/L3/L4)
 - Kazama et al., EPS, 2017

tplot変数のリストを表示 & tplot変数の中身を見る

```
pyspedas.tplot_names()
```

```
data = pyspedas.getdata('erg_mgf_l2_mag_8sec_gsm', xarray = True)
```

```
print(data)
```

```
0 : erg_mgf_l2_epoch_8sec
1 : erg_mgf_l2_mag_8sec_dsi
2 : erg_mgf_l2_mag_8sec_gse
3 : erg_mgf_l2_mag_8sec_gsm
4 : erg_mgf_l2_mag_8sec_sm
5 : erg_mgf_l2_magt_8sec
6 : erg_mgf_l2_rmsd_8sec_dsi
7 : erg_mgf_l2_rmsd_8sec_gse
8 : erg_mgf_l2_rmsd_8sec_gsm
9 : erg_mgf_l2_rmsd_8sec_sm
10 : erg_mgf_l2_rmsd_8sec
11 : erg_mgf_l2_n_rmsd_8sec
12 : erg_mgf_l2_dyn_rng_8sec
13 : erg_mgf_l2_quality_8sec
14 : erg_mgf_l2_quality_8sec_gc
15 : erg_mgf_l2_igrf_8sec_dsi
16 : erg_mgf_l2_igrf_8sec_gse
17 : erg_mgf_l2_igrf_8sec_gsm
18 : erg_mgf_l2_igrf_8sec_sm
```

通し番号付きでロード済みの
tplot変数のリストが表示され
る

```
<xarray.DataArray 'erg_mgf_l2_mag_8sec_gsm' (time: 53555, v_dim: 3)> Size: 643kB
array([[ 135.60878 , -320.36      ,  19.091593],
        [ 135.57445 , -319.85028 ,  19.290548],
        [ 135.61684 , -319.40616 ,  19.280993],
        ...,
        [ 2167.1052  , -16620.145   ,  8978.857   ],
        [ 2328.3225  , -16931.568   ,  8880.529   ],
        [ 2496.911   , -17243.72    ,  8775.3125  ]],
      shape=(53555, 3), dtype=float32)
Coordinates:
  * time      (time) datetime64[ns] 428kB 2023-04-21T23:59:59.370838016 ... 20...
Dimensions without coordinates: v_dim
Attributes:
  CDF:      {'VATT': {'FIELDNAM': 'B in GSM', 'CATDESC': '8 sec resolu...
  data_att: {'coord_sys': '', 'units': 'nT', 'depend_1_units': 'nT', '...
  plot_options: {'xaxis_opt': {'axis_label': '', 'crosshair': 'X', 'x_axis...
```

tplot変数 'erg_mgf_l2_mag_8sec_gsm' に紐づけられている実際のデータ
構造体の情報が表示される。

*.values: 実際の時系列データの配列、ここでは 53555 x 3 要素の2次元配列

tplot変数のリストを表示 & tplot変数の中身を見る

```
data = pyspedas.getdata('erg_mgf_l2_mag_8sec_gsm', xarray = True)
```

```
for i in data.CDF['GATT'].items():
```

```
    print(i)
```

```
(('PROJECT', ['ERG>Exploration of Energization and Radiation in Geospace']))
(('DISCIPLINE', ['Space Physics>Magnetospheric Science']))
(('SOURCE_NAME', ['ARASE(ERG)>Inner Magnetosphere']))
(('DATA_TYPE', ['l2_mgf>level 2 spin-averaged magnetic field data']))
(('DESCRIPTOR', ['MGF>Magnetic Field Experiment']))
(('DATA_VERSION', ['04', '06']))
(('DOI', ['10.34515/DATA.ERG-06001']))
(('TITLE', ['Level 2 magnetic field data obtained by the Magnetic Field Experiment (MGF)
instrument onboard the ERG satellite']))
(('TEXT', ['IDL Geopack v11.0 and IGRF-14 were used to convert the coordinate systems of the
magnetic field and calculate the IGRF-model field.']))
(('GENERATED_BY', ['ERG Science Center, operated by ISAS/JAXA and ISEE/Nagoya University as a
Joint Research Center for Space Science']))
(('GENERATION_DATE', ['20251211']))
(('MODS', ['Created 12/2025']))
(('ADID_REF', [' ']))
(('LOGICAL_FILE_ID', ['erg_mgf_l2_8sec_20230426_v04.06']))
(('LOGICAL_SOURCE', ['erg_mgf_l2_8sec']))
(('LOGICAL_SOURCE_DESCRIPTION', ['Exploration of Energization and Radiation in Geospace (ERG)
Magnetic Field Experiment (MGF) Level 2 spin-averaged magnetic field data']))
(('PI_NAME', ['Ayako Matsuoka']))
(('PI_AFFILIATION', ['Data Analysis Center for Geomagnetism and Space Magnetism, Graduate
School of Science, Kyoto University, Kitashirakawa-Oiwake Cho, Sakyo-ku Kyoto 606-8502,
Japan']))
(('MISSION_GROUP', ['ERG']))
(('INSTRUMENT_TYPE', ['Magnetic Fields (space)']))
(('TEXT_SUPPLEMENT', [' ']))
((('RULES_OF_USE', ['Rules of the Road for the users of the products from the ERG/Arase
Project ', ' 1. Users of all level of scientific products from the ERG/Arase project should
contact instrument PI team(s), Project Manager (PM), and Project Scientist (PS) before using
the data for any presentation and publication. PI(s) / PM (Iku Shinohara, iku at stp.isas.
jaxa.jp) / PS (Yoshizumi Miyoshi, miyoshi at isee.nagoya-u.ac.jp) may suggest potential
```

メタデータ (科学データに関する各種情報) が表示される

RULES_OF_USE という要素に **"rules of the road"** (データを使う・発表する際に守るべきルール)が入っている。

Rules of the road (データポリシー)



PI: Ayako Matsuoka

Affiliation: Institute of Space and Astronautical Science, Japan
Aerospace Exploration Agency, 3-1-1 Yoshinodai, Chuo-ku, Sagami-hara,
Kanagawa 252-5210, Japan

Rules of the Road for the users of the products from the ERG/Arase Project

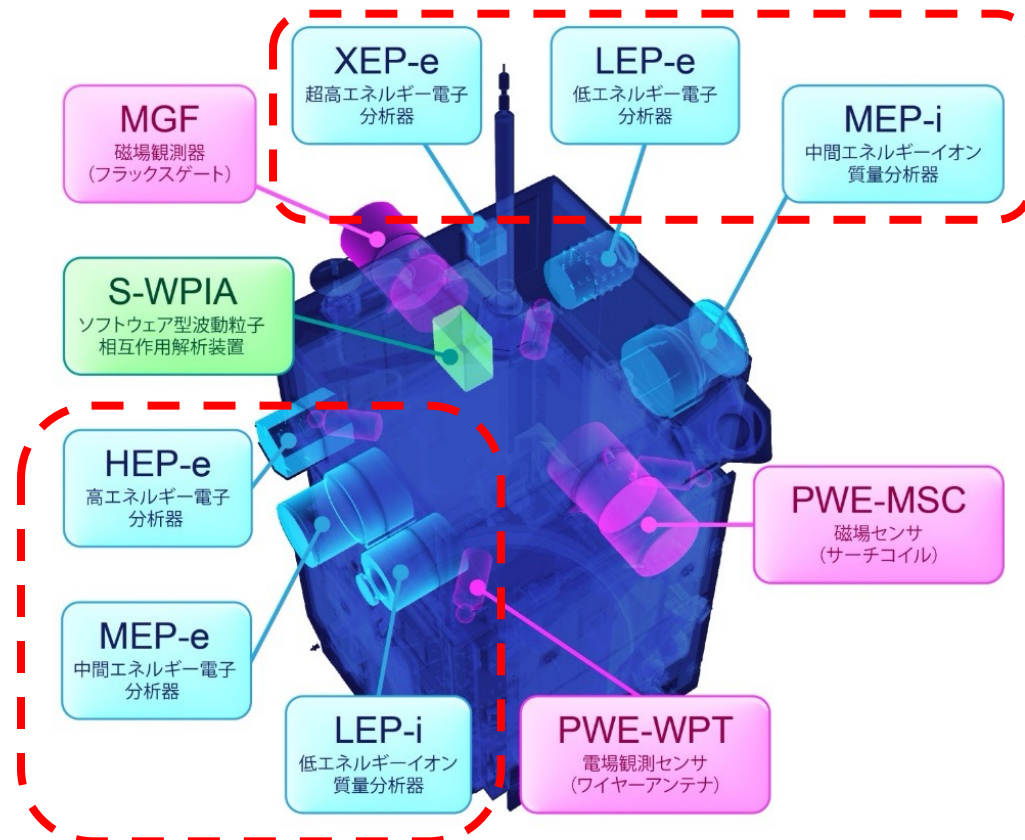
1. Users of all level of scientific products from the ERG/Arase project should contact instrument PI team(s), Project Manager (PM), and Project Scientist (PS) before using the data for any presentation and publication. PI(s) / PM (Iku Shinohara, iku at stp.isas.jaxa.jp) / PS (Yoshizumi Miyoshi, miyoshi at isee.nagoya-u.ac.jp) may suggest potential coauthor(s) from the ERG/Arase project side for the presentation and publication. Some necessary articles suggested by the PI(s) should be cited.
 2. Users should always use the latest version of data files in CDF provided from the ERG science center for their data analysis, presentation and publications. Redistribution of the data files is strictly prohibited.
 3. Users should send presentation materials and papers including ERG/Arase data to instrument PI team(s), PM, and PS enough before presentation and paper submission, so that sufficient time is available for those responsible for the data to check if the data are properly processed/used and to get necessary comments back to the data users.
 4. Publications that use ERG/Arase satellite data should cite the project overview paper (Miyoshi et al., Earth Planets Space, DOI:10.1186/s40623-018-0862-0, 2018) and include the following text in the paper acknowledgements: "Science data of the ERG (Arase) satellite were obtained from the ERG Science Center operated by ISAS/JAXA and ISEE/Nagoya University (<https://ergsc.isee.nagoya-u.ac.jp/index.shtml.en>)."
- Rules of the Road of Level-2 data (MGF)
The MGF Level-2 data should be used based on a full understanding of the measurement limit. The data are produced to achieve the accuracy

- データを解析等に使う人は全員このルールを守らないといけない。
- 変なデータを見つけた場合は機器PIチームに報告することが推奨されている (erg_xxx_info@isee.nagoya-u.ac.jp)。

- MEPIの'erg_mepi_l2_omniflux_FPDO'と、PWE-EFDの'erg_pwe_efd_l2_spec_spectra'をxarray形式の配列で取得しましょう。また、取得した配列それぞれの物理量を確認してみましょう。
 - ✓ データの単位は、'data_att'の中に'units'というkeyで格納されています。
- MGFのうち、'erg_mgf_l2_mag_8sec_gse', 'erg_mgf_l2_mag_8sec_gsm'を配列で取得し、それぞれの絶対値を計算してみましょう。また、同時に'erg_mgf_l2_magt_8sec'も配列で取得し、matplotlibでそれぞれの絶対値の値をプロットし比較してみましょう。
 - ✓ 絶対値の計算には、`np.linalg.norm()`を使うと便利です。

応用パート

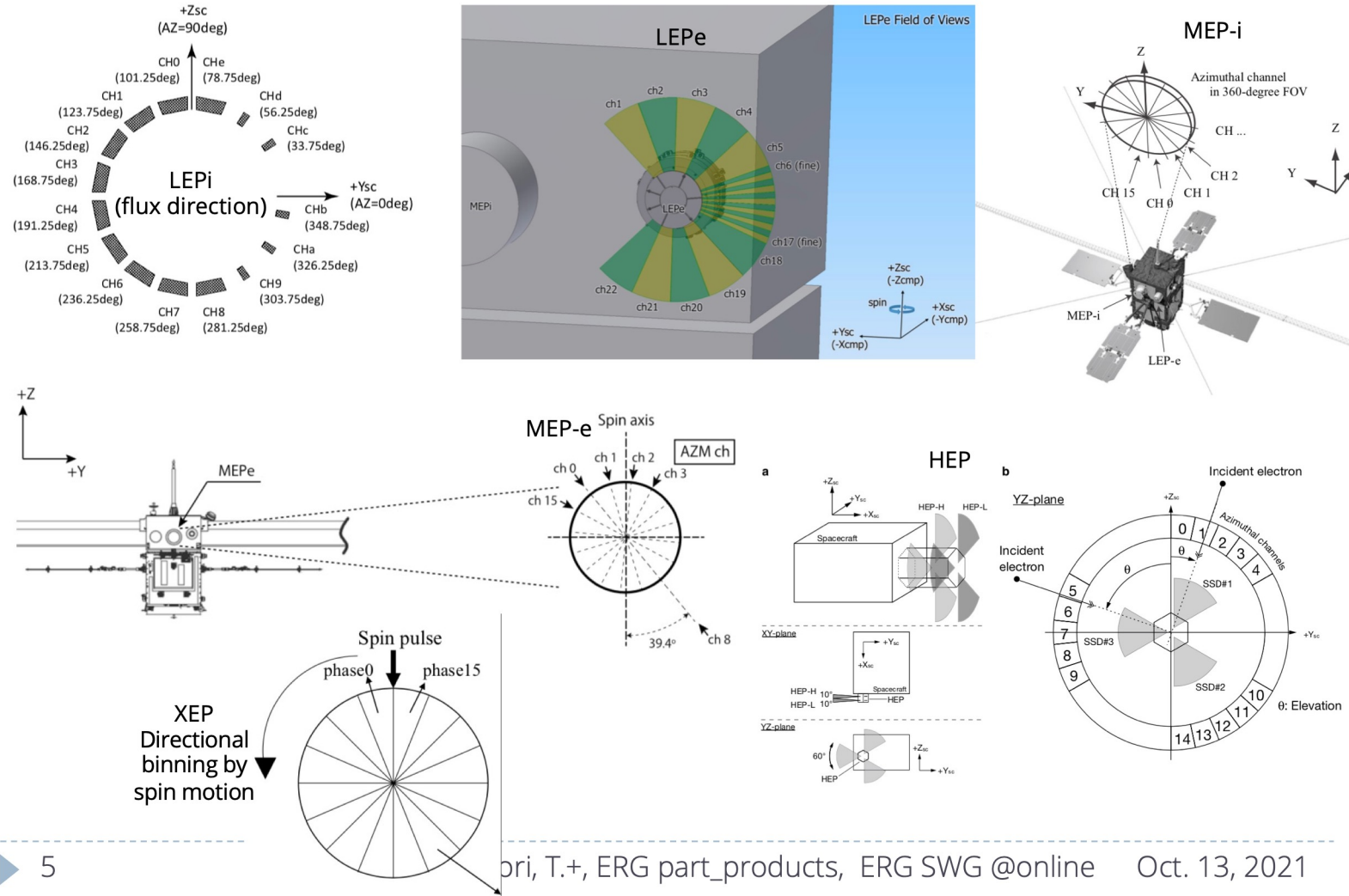
part_productsライブラリを使った 粒子データの解析



- XEP (L2/L3/L4)
 - Lv.2 2-D electron flux data (DOI: 10.34515/DATA.ERG-00000)
- HEP (HEP-H/HEP-L, pre/ L2/L3/L4)
 - Mitani et al., EPS, 2018
 - Lv.2 3-D electron flux data (DOI: 10.34515/DATA.ERG-01000)
- MEPI(pre/L2/L3/L4)
 - Yokota et al., EPS, 2017
 - Lv.2 3-D ion flux data (DOI: 10.34515/DATA.ERG-03000)
- MEPE(pre/L2/L3/L4)
 - Kasahara et al., EPS, 2018
 - Lv.2 3-D electron flux data (DOI: 10.34515/DATA.ERG-02000)
- LEPI(pre/L2/L3/L4)
 - Asamura et al., EPS, 2018
 - Lv.2 3-D ion flux data (DOI: 10.34515/DATA.ERG-05000)
- LEPE(pre/L2/L3/L4)
 - Kazama et al., EPS, 2017
 - Lv.2 3-D electron flux data (DOI: 10.34515/DATA.ERG-04001)

Quick reference for the field-of-views of the particle instruments

Please see the instrument papers in the prev. page for the details



Arase衛星の粒子データ: まずはOMNI flux

```
trange = ['2023-04-24/07:40:00', '2023-04-24/11:40:00']
```

```
var_** = **(trange=trange, datatype='omniflux')
```

** ← xep, hep, mepi_nml, mepe, lepi, lepeを指定することで
任意の測器のomni flux データをロードできる。

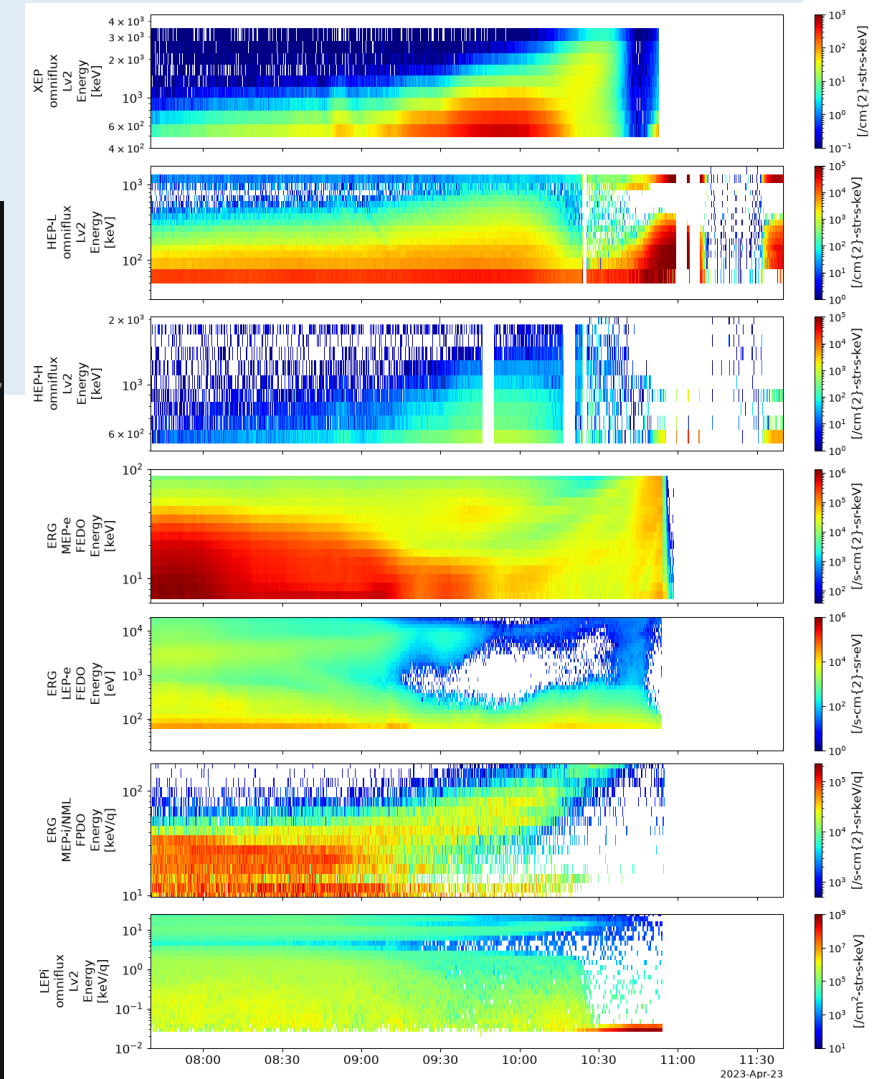
```
['erg_xep_l2_FED0_SSD',
'erg_hep_l2_FED0_L',
'erg_hep_l2_FED0_H',
'erg_mepi_l2_omniflux_epoch',
'erg_mepi_l2_omniflux_epoch_tof',
'erg_mepi_l2_omniflux_FIDO_Energy',
'erg_mepi_l2_omniflux_FPDO',
'erg_mepi_l2_omniflux_FHE2D0',
'erg_mepi_l2_omniflux_FHED0',
'erg_mepi_l2_omniflux_FOPPDO',
'erg_mepi_l2_omniflux_FODO',
'erg_mepi_l2_omniflux_FO2PDO',
'erg_mepi_l2_omniflux_FPDO_tof',
'erg_mepi_l2_omniflux_FHE2D0_tof',
'erg_mepi_l2_omniflux_FHED0_tof',
'erg_mepi_l2_omniflux_FOPPDO_tof',
'erg_mepi_l2_omniflux_FODO_tof',
'erg_mepi_l2_omniflux_FO2PDO_tof',
'erg_mepe_l2_omniflux_epoch',
'erg_mepe_l2_omniflux_FED0',
'erg_lepi_l2_omniflux_Epoch',
'erg_lepi_l2_omniflux_FPDO_raw',
'erg_lepi_l2_omniflux_FHED0_raw',
'erg_lepi_l2_omniflux_FODO_raw',
'erg_lepi_l2_omniflux_FPDO',
'erg_lepi_l2_omniflux_FHED0',
'erg_lepi_l2_omniflux_FODO',
'erg_lepe_l2_omniflux_FED0']
```

Arase衛星の粒子データにおけるOMNI fluxは
様々な方向から到来した粒子フラックスを方向について
積分し、**算術平均を計算したもの**。

「あるエネルギーの粒子がどの程度存在するか」を示す

****ピッチアングル平均したfluxとは異なることに注意**

$$omniflux(E) = \frac{1}{N} \sum_{dir.ch}^N flux(dir, E) \neq \frac{1}{\pi} \int_0^{\pi} d\alpha \cdot flux(\alpha, E),$$





Arase衛星の粒子データ: 3-D / 2-D flux data

```
trange = ['2023-04-24/07:40:00', '2023-04-24/09:20:00' ]
```

```
var_** = **(trange=trange, datatype= '3dflux' )
```

** ← hep, mepi_nml, mepe, lepi, lepeを指定すること任意の測器の3次元分布関数データをロードできる。

ただし、xepについては2dfluxとなることに注意

3-D / 2-D flux dataは3次元よりも高い次元の配列であるため、そのままではプロットできない

→ Part productsを使って情報の次元を落としていく
(ある次元について積分していく)

```
0 : erg_xep_l2_FEDU_SSD
1 : erg_hep_l2_FEDU_L
2 : erg_hep_l2_FEDU_H
3 : erg_hep_l2_FEDU_L_Angle_sga
4 : erg_hep_l2_FEDU_H_Angle_sga
5 : erg_hep_l2_FEDU_L_Angle_gse
6 : erg_hep_l2_FEDU_H_Angle_gse
7 : erg_hep_l2_sctno_L
8 : erg_hep_l2_sctno_H
9 : erg_hep_l2_rawcnt_H
10 : erg_hep_l2_rawcnt_L
...
81 : erg_lepe_l2_3dflux_count_rate
82 : erg_lepe_l2_3dflux_count_rate_bg
83 : erg_lepe_l2_3dflux_energy_index
84 : erg_lepe_l2_3dflux_FEDU
```

"part_products"とは何か?

- 粒子データの各種スペクトルプロットを作成するのに使用する、SPEDASに集録されているルーチン群のこと
- ERG-SCの提供する LEP-e, LEP-i, MEP-e, MEP-i, HEP, and XEP dataで使用可能。
 - ▶ `ergpypedas.erg_xep_part_products` : XEP
 - ▶ `ergpypedas.erg_hep_part_products` : HEP
 - ▶ `ergpypedas.erg_mep_part_products` : MEP-e and MEP-i
 - ▶ `ergpypedas.erg_lep_part_products` : LEP-e and LEP-i



part_productsの基本: 平均フラックスのエネルギースペクトル



```
trange = ['2023-04-24/07:40:00', '2023-04-24/11:40:00']
```

…3dfluxデータをロード…

```
var_hep_et = erg_hep_part_products('erg_hep_l2_FEDU_L', outputs='energy', trange=trange)
```

```
var_mepe_et = erg_mep_part_products('erg_mepe_l2_3dflux_FEDU', outputs='energy', trange=trange)
```

```
var_lepe_et = erg_lepe_part_products('erg_lepe_l2_3dflux_FEDU', outputs='energy', trange=trange)
```

```
tplot(['erg_hep_l2_FEDU_L_energy', 'erg_mepe_l2_3dflux_FEDU_energy', 'erg_lepe_l2_3dflux_FEDU_energy'])
```

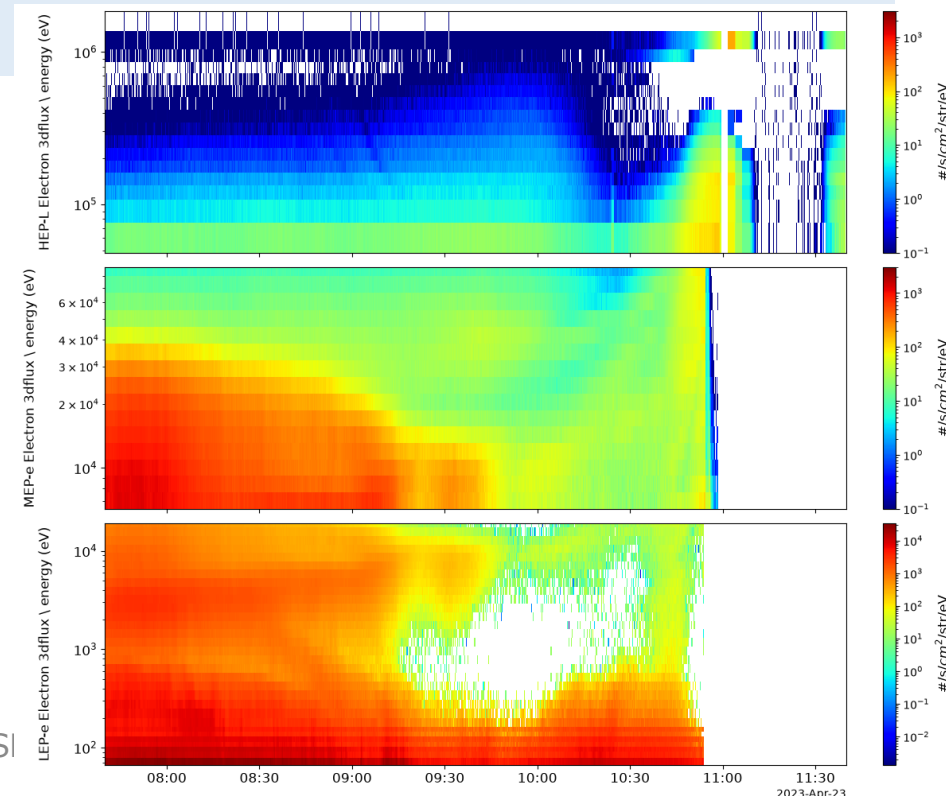
outputs= 'energy' を指定すると**_energyというtplot変数が取得できる

時間 x エネルギー x 方位角 x 仰角



時間 x エネルギー

(方位・仰角方向に粒子フラックスを全部平均)



part_productsの基本: あるエネルギー範囲でのpitch角スペクトル



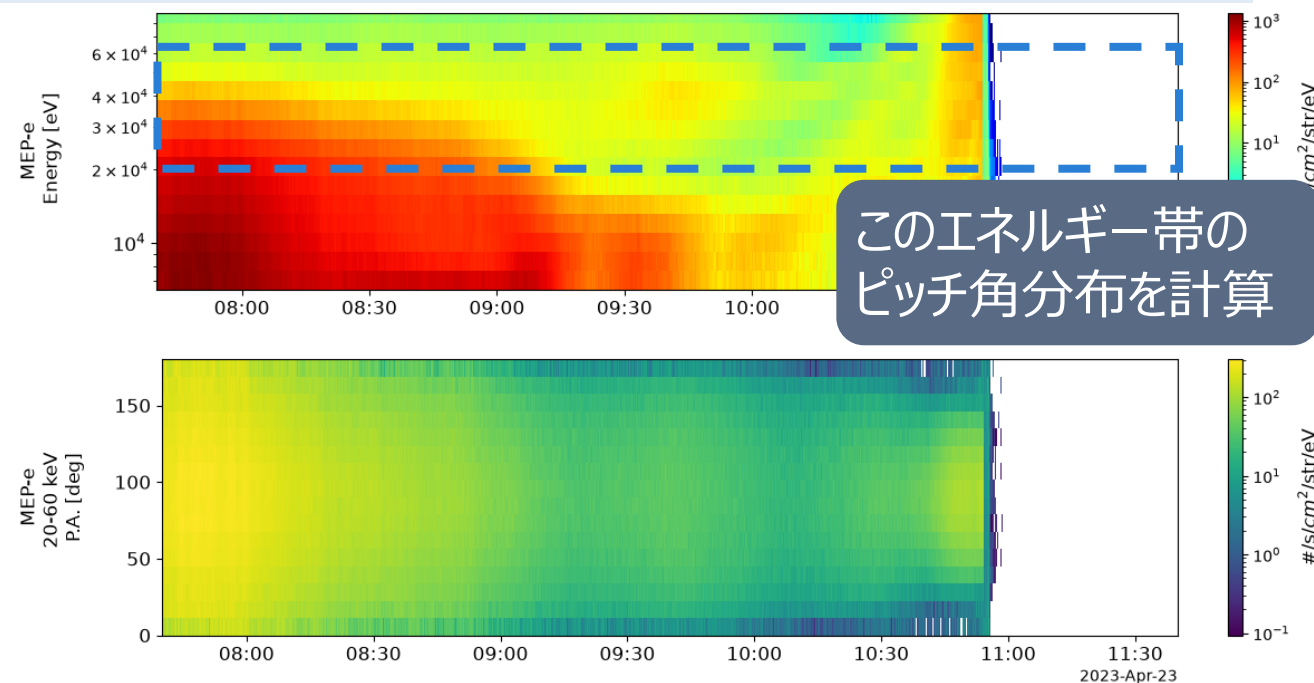
```
trange = ['2023-04-24/07:40:00', '2023-04-24/11:40:00']
var_orb = orb(trange=trange)
var_mgf = mgf(trange=trange)
var_mepe = mepe(trange=trange, datatype='3dflux')
var_mepe_pad = erg_mep_part_products( 'erg_mepe_l2_3dflux_FEDU' , energy=[20000., 60000], # 計算するエネルギー幅
                                     mag_name= 'erg_mgf_l2_mag_8sec_dsi' , # DSI座標系の磁場データ
                                     pos_name= 'erg_orb_l2_pos_gse' , # GSE座標系での軌道データ
                                     outputs='pa', trange=trange)

tplot(['erg_mepe_l2_3dflux_FEDU_energy', 'erg_mepe_l2_3dflux_FEDU_pa'])
```

時間 x エネルギー x 方位角 x 仰角

↓
時間 x pitch角

あるエネルギー範囲のフラックスだけを取り出す。
磁場データを考慮して方位角x仰角を粒子のpitch角に
なおして、pitch角毎に平均フラックスを出す。



part_productsの基本: あるピッチ角範囲のフラックス値のエネルギースペクトル

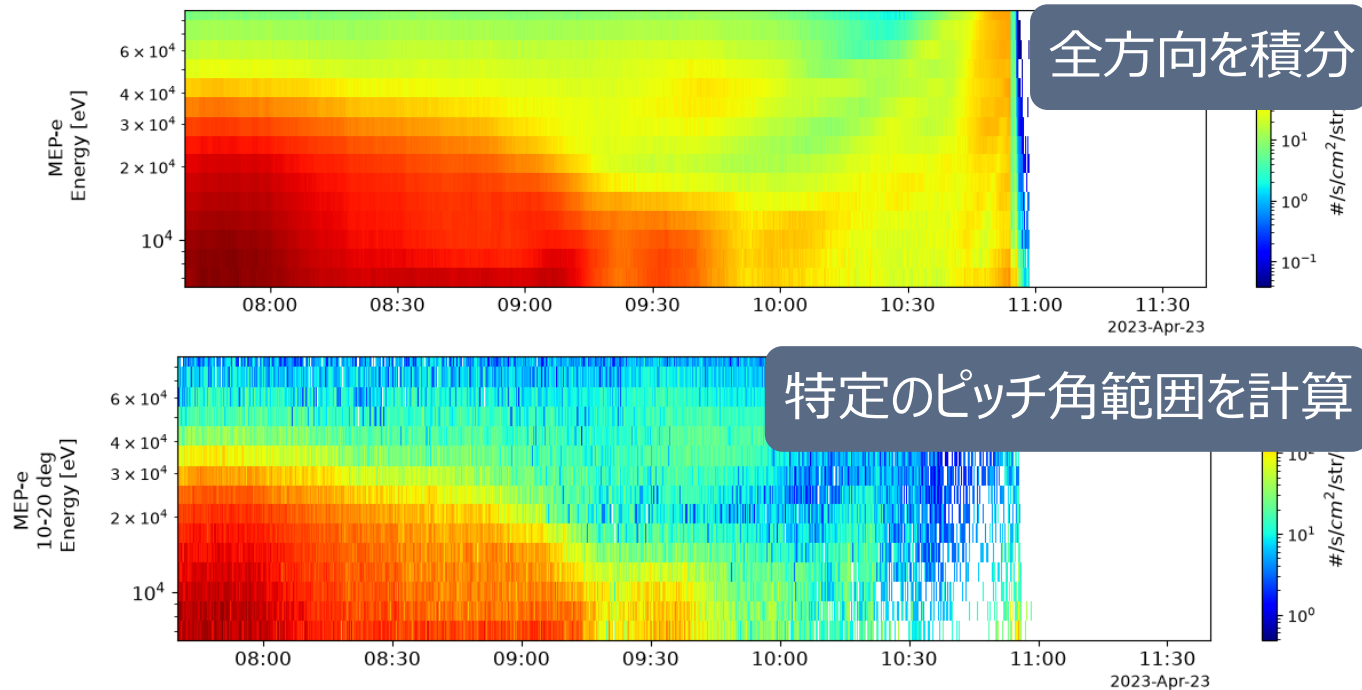
```
var_mepe_et = erg_mep_part_products('erg_mepe_l2_3dflux_FEDU' ,
    pitch=[10,20], # 計算するピッチ書くの範囲
    mag_name='erg_mgf_l2_mag_8sec_dsi', # DSI座標系の磁場データ
    pos_name='erg_orb_l2_pos_gse', # GSE座標系での軌道データ
    outputs='energy', trange=trange)

tplot(['erg_mepe_l2_3dflux_FEDU_energy_mag'], ysize = 3)
```

時間 x エネルギー x 方位角 x 仰角
↓
時間 x エネルギー

方位角と仰角からピッチ角を算出し、そのピッチ角が「ある範囲」に入っているフラックス値のみ抜き出して、そのエネルギースペクトルを求めている。

上記の例ではピッチ角が 10° – 20° の範囲を指定している (`pitch=[10, 20]`)





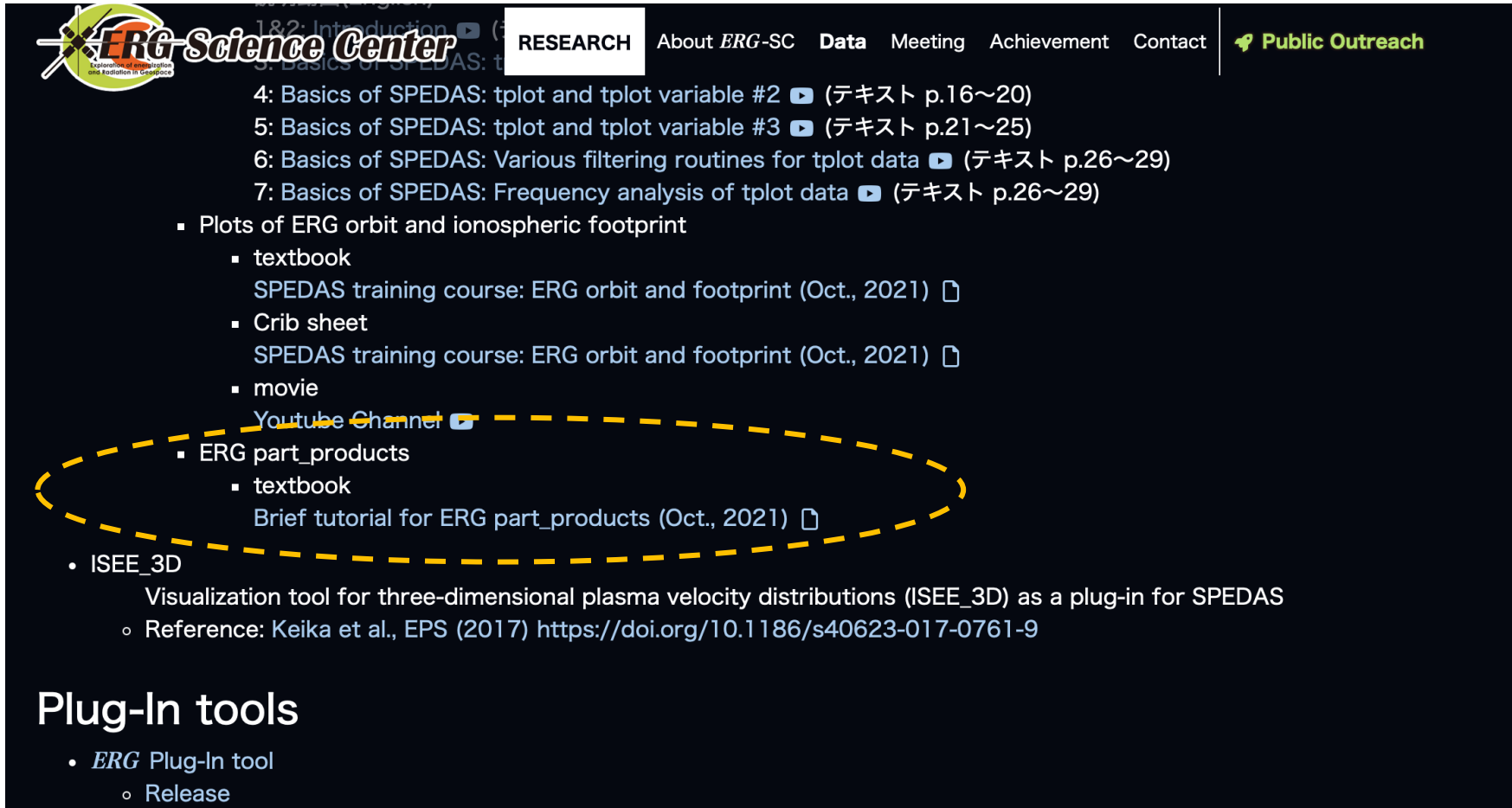
- `pyspedas.options('var_name', 'option', **)`をつかうとtplotによるプロットのパネルを装飾できる。
選択できるoptionは沢山あるので、詳細は省くが、過去の講習会資料やPySPEDASの公式Githubにカスタマイズ方法が紹介されている。
- 過去の講習会資料:
https://github.com/ergsc-devel/arase_tutorial/blob/devel2/PySPEDAS_Training_Course_ERG_SWG_20260304.ipynb
- PySPEDASのGithub: <https://pyspedas.readthedocs.io/en/latest/plotting.html>

- LEP-eの3dfluxデータを使用して、エネルギースペクトルを計算してみましょう。
- さらに、ピッチ角範囲 $[0,45]$, $[45,90]$, $[90,135]$, $[135,180]$ のフラックス値のエネルギースペクトルをそれぞれ計算し、その重ね合わせが、全方向のフラックスのエネルギースペクトルと一致することを確認してください。
- また、ピッチ角スペクトルもプロットし、ピッチ角分布とエネルギースペクトルの関係を確認してみましょう。

もっと詳しい情報は...

- ERG-SCウェブページに、もう少し詳しいpart_productsの使い方に関するスライドがある(ただし英語):

https://ergsc.isee.nagoya-u.ac.jp/data_info/howto.shtml.en



ERG Science Center | RESEARCH | About ERG-SC | Data | Meeting | Achievement | Contact | Public Outreach

- 4: Basics of SPEDAS: tplot and tplot variable #2 (テキスト p.16~20)
- 5: Basics of SPEDAS: tplot and tplot variable #3 (テキスト p.21~25)
- 6: Basics of SPEDAS: Various filtering routines for tplot data (テキスト p.26~29)
- 7: Basics of SPEDAS: Frequency analysis of tplot data (テキスト p.26~29)
- Plots of ERG orbit and ionospheric footprint
 - textbook
 - SPEDAS training course: ERG orbit and footprint (Oct., 2021)
 - Crib sheet
 - SPEDAS training course: ERG orbit and footprint (Oct., 2021)
 - movie
 - Youtube Channel
 - ERG part_products
 - textbook
 - Brief tutorial for ERG part_products (Oct., 2021)
- ISEE_3D
 - Visualization tool for three-dimensional plasma velocity distributions (ISEE_3D) as a plug-in for SPEDAS
 - Reference: Keika et al., EPS (2017) <https://doi.org/10.1186/s40623-017-0761-9>

Plug-In tools

- ERG Plug-In tool
 - Release

応用パート 磁場データの時系列解析

1. 算術平均・移動平均の計算
2. 配列同士の演算
3. データの補間
4. フィルターの適用
5. 周波数解析

Pythonはnumpy、scipyなど便利な解析パッケージが存在

1. データのロード+データ整理まではPySPEDASで行い、
詳細な解析は自身のプログラムで行う

2. データのロードから解析までを全てPySPEDASで行う

PySPEDASを使った解析のメリット

- コミュニティで共通のルーチンで解析が行える
- 特定の用途に特化した関数がある

PySPEDASを使った解析のデメリット

- 解析プログラムのアップデート等、変更を追う必要あり

時系列データ解析の基本操作：算術平均・移動平均の計算



```
pyspedas.avg_data('varname', res = 60)
```

varname: tplot変数名 (文字列、整数)

res : 平均する時間幅 (単位は秒、defaultは60秒)

****widthで平均するデータ点数も指定できるが、2026年9月時点では動作が不安定**

```
pyspedas.tsmooth('varname', width = 10)
```

varname: tplot変数名

width : 平均するデータ点数 (defaultは10点)

```
trange = ['2023-04-23/07:40:00', '2023-04-24/07:40:00']
```

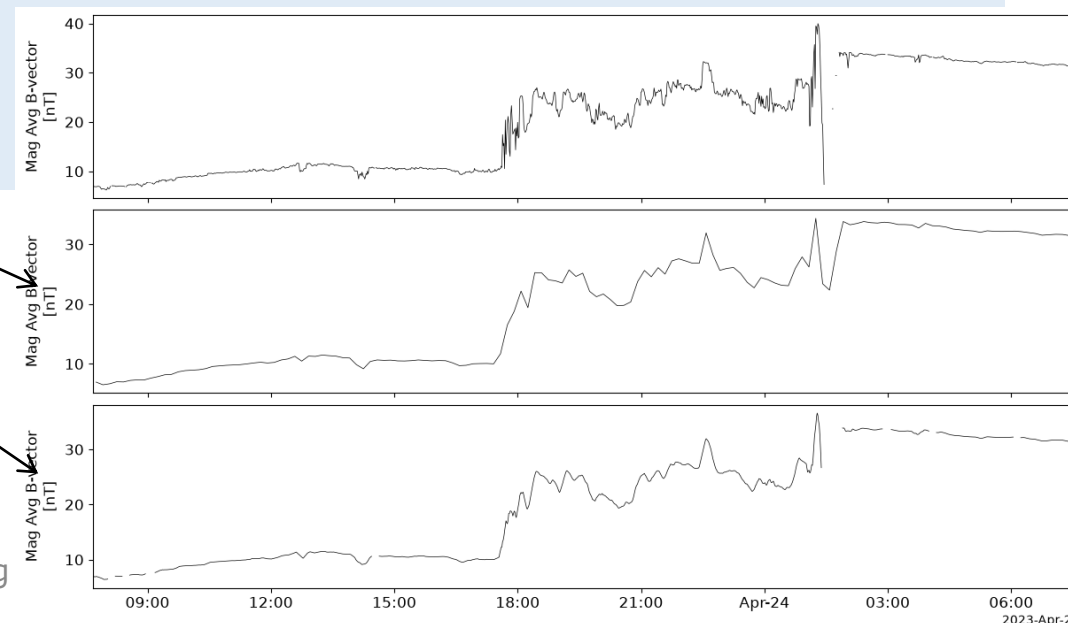
```
var_omni = pyspedas.projects.omni.load(trange=trange, level='hro2')
```

```
pyspedas.avg_data('F', res = 600)
```

```
pyspedas.tsmooth('F', width = 10)
```

```
pyspedas.tplot(['F', 'F-avg', 'F-s'])
```

- avg_data: 600秒間隔でデータを平均
‘*-avg’というtplot変数が新たに作られる
****元は1分ごとに値があったものが、10分に1つになって、結果としてデータ点数が減っている**
- tsmooth: 10点ごとのデータで移動平均を計算
‘*-s’というtplot変数が新たに作られる
****データ点の数はフィルター前と変わらない。**



時系列データ解析の基本操作：四則演算



```
pyspedas.add('varname')
```

varname: tplot変数名 (文字列、整数)

add, subtract, multiply, divide: tplot変数同士の四則演算ができる

```
trange = ['2023-04-24', '2023-04-25']
```

```
var_orb = orb(trange=trange)
```

```
var_mgf = mgf(trange=trange)
```

```
Bmodel = tt89("erg_orb_l2_pos_gsm", iopt=1, units_in="Re", coord_in="GSM", coord_out="GSM", igrf_only=False)
```

```
subtract('erg_mgf_l2_mag_8sec_gsm', 'erg_orb_l2_pos_gsm_bt89', newname='dif_Bgsm')
```

```
tplot(['erg_mgf_l2_mag_8sec_gsm', 'erg_orb_l2_pos_gsm_bt89', 'dif_Bgsm'])
```

Arase衛星のmgfデータから磁場モデル(T89)を差し引いて、
磁場の擾乱を計算する

```
pyspedas.geopack.tt89
```

```
tt89('varname',
```

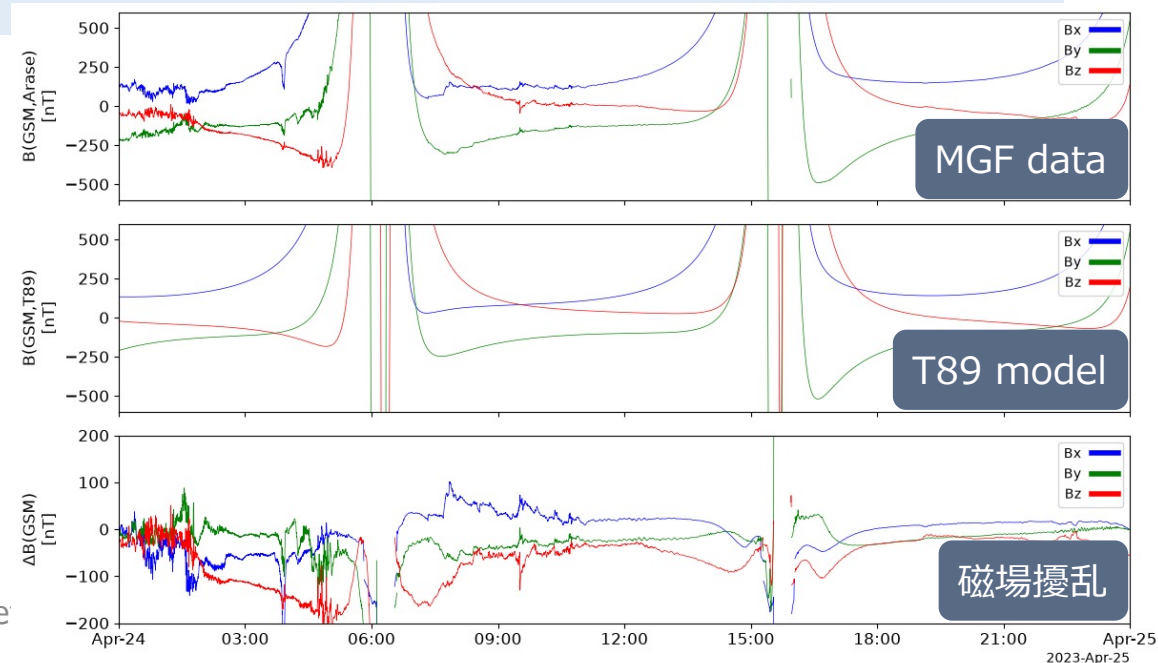
```
    iopt=1, # ipot: 1:Kp=0,0+, 2:Kp=1-,1,1+, ...
```

```
    units_in="Re", # 軌道データの単位("km" or "Re")
```

```
    coord_in="GSM", # 入力の座標系
```

```
    coord_out="GSM", # 出力の座標系
```

```
    igrf_only=False) # Trueの場合、磁場の計算にIGRF modelを使用
```



時系列データ解析の基本操作：ダイナミックスペクトル – tdpwrspc–



```
pyspedas.tdpwrspc('varname')
```

varname : tplot変数名

```
pyspedas.tdpwrspc('dif_Bgsm')
```

```
options('*_dpwrspc','colormap', 'viridis')
```

```
options('dif_Bgsm_z_dpwrspc','ytitle', r'$\Delta$ Bz(SM)')
```

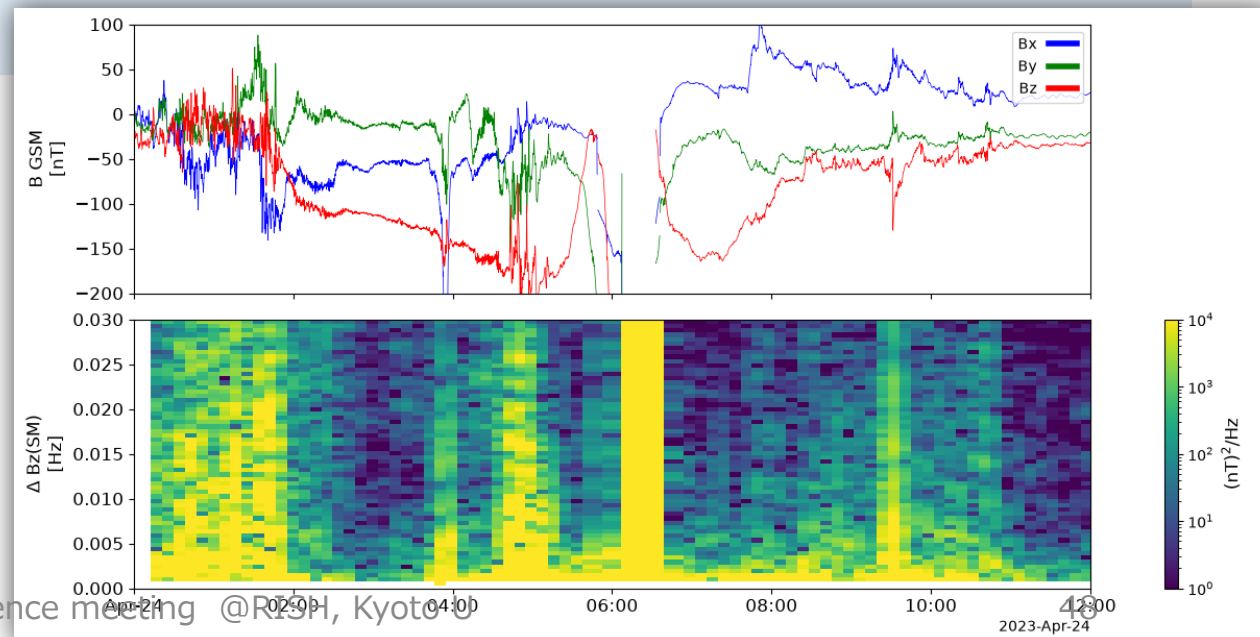
```
options('*_dpwrspc','ylog',0)
```

```
options('*_dpwrspc','zrange',[1.0e0,1.0e4])
```

```
options('*_dpwrspc','yrange',[0,0.03])
```

```
tplot(['dif_Bgsm', 'dif_Bgsm_z_dpwrspc'], ysize = 6)
```

高速フーリエ変換(FFT) + Hanning窓 を
使って、上の時系列データの周波数スペクトルを求めている。



- 磁場データを`get_data`で取得し、`numpy` or `scipy`を使って移動平均を計算してみましょう。(ローパス処理)
データの期間は `trange = ['2023-04-24', '2023-04-24/06:00']`
移動平均の時間幅はデータ100点としてください (800秒)
 - ✓ `numpy`を使った移動平均は、`np. ones`と`np.convolve`, `pandas`を使った移動平均は`pandas.DataFrame.rolling`, `scipy`を使った移動平均は`scipy.signal.savgol_filter`をよく使います。詳しくはノートブックのヒントを参照してください。
- 計算した移動平均を元のデータから差し引いて、元のデータと比較したプロットを作成してみましょう。(ハイパス処理)
- 移動平均を差し引いた磁場の擾乱データについて`pyspedas.wavelet98`を使ってwavelet解析をしてみましょう。(使い方はnotebookのヒントを参照してください)
 - ✓ Wavelet解析：周波数解析の代表的な手法の一つ。様々な波長のmother wavelet(波束)を畳み込むことで、時系列データにうめこまれた様々なスケールの変動について、時間変動、周波数変動を同時に追跡できる。

応用パート その他の便利な関数

ベクトル値入りのtplot変数を成分ごとに分離・結合

```
pyspedas.split_vec('erg_mgf_l2_mag_8sec_sm')
```

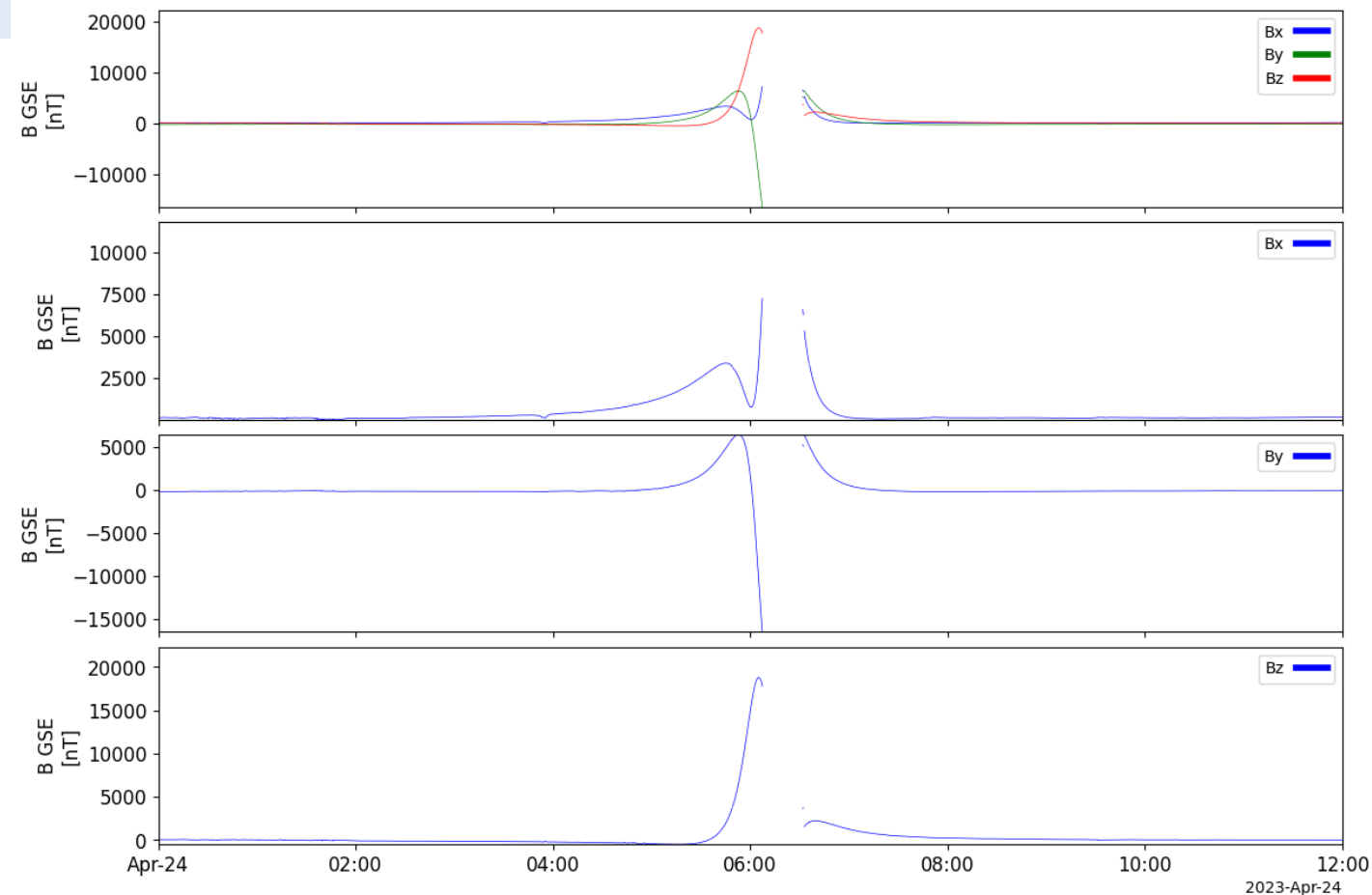
→ 出力のtplot変数 `erg_mgf_l2_mag_8sec_sm_x`, `erg_mgf_l2_mag_8sec_sm_y`, `erg_mgf_l2_mag_8sec_sm_z`

```
tplot(['erg_mgf_l2_mag_8sec_sm_x', 'erg_mgf_l2_mag_8sec_sm_y', 'erg_mgf_l2_mag_8sec_sm_z'])
```

`split_vec` / `join_vec` はベクトル値や配列の時系列データを格納するtplot変数を、各成分ごとに別々のtplot変数にする。

→ 別のパネルにして並べてプロットできる!

→ 自分で独立に処理した各成分を結合して新たなベクトルを作れる。



座標変換に役立つツール： FAC, MVA, LMN座標への変換



```
var_orb = orb(trange=trange)
```

```
var_mag = mag(trange=trange)
```

… 'erg_mgf_l2_mag_8sec_gse' の座標変換をする場合は前処理でNaNを除去する …

```
pyspedas.fac_matrix_make('var_name', pos_var_name='erg_orb_l2_pos_gse')
```

```
pyspedas.tvector_rotate('var_name_fac_mat', 'erg_mgf_l2_mag_8sec_gse')
```

```
tplot(['erg_mgf_l2_mag_8sec_gse_rot'])
```

座標変換の手順

```
pyspedas.fac_matrix_make
```

```
pyspedas.mva_matrix_make
```

```
pyspedas.lmn_matrix_make
```

で変換行列を作成する。

次に、

```
pyspedas.tvector_rotate
```

で座標変換(ベクトルの回転)を行う

